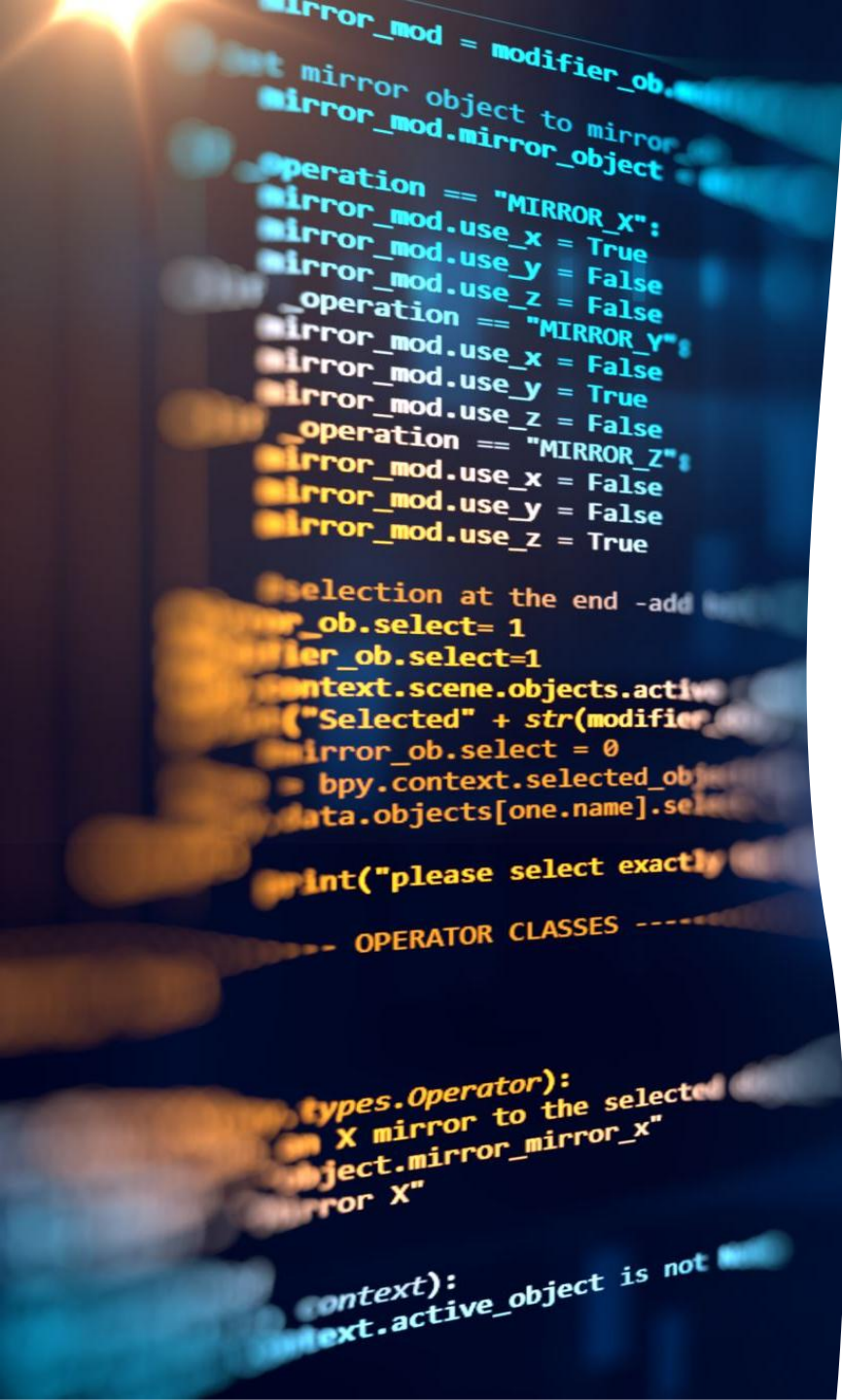




TensorFlow Lite Micro: Architecture Decisions

* During Interpretation, the runtime reads and executes a model **directly from a serialized format** (FlatBuffers) without generating new source code or recompiling the project for every model. [Read more on Flatbuffers here.](#)

Interpretation vs Code Generation

Choice: Model interpretation rather than code generation

Benefits of Interpretation:

- **Upgradability** - easy framework updates
- **Multiple models** support without code duplication
- **Model replacement** without recompilation
- **Inline data** from FlatBuffers format *

Project Generation Approach:

- **Copy required source files** without modification
- **IDE-specific project files** for easy building
- **External dependencies** included in project
- **Upgradable** through standard merge tools

Memory Management in TinyML

Static Memory Allocation Strategy:

- No `malloc()/free()` to avoid heap fragmentation
- Pre-planned memory layout for long-running applications
- Fixed-size arena passed at initialization
- Immediate error if arena too small

Memory Arena Concept:

```
// Application provides memory arena
constexpr int kTensorArenaSize = 60 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

// Framework uses arena for all allocations
tflite::MicroInterpreter interpreter(model, resolver,
                                     tensor_arena, kTensorArenaSize);
```

uint8_t is a typedef in C and C++ that represents an unsigned 8-bit integer

Benefit: Predictable memory usage, no runtime allocation failures

Model File Format: FlatBuffers

FlatBuffers is a binary serialization format plus a schema compiler. You define a schema (what fields exist), then serialize data into a flat binary layout that can be read directly without parsing or allocating. Readers access fields via pointers/offsets—no unpacking step.

Why FlatBuffers?

- Zero-copy deserialization - direct memory access
- Same in-memory and serialized format
- Embedded directly in flash memory
- No parsing overhead

Schema Structure (schema.fbs):

```
table Model {  
  version: uint;  
  operator_codes: [OperatorCode];  
  subgraphs: [SubGraph];  
  buffers: [Buffer];  
}  
  
table SubGraph {  
  tensors: [Tensor];  
  operators: [Operator];  
  inputs: [int];  
  outputs: [int];  
}  
  
root_type Model;
```

Model Loading Example

Loading a Model:

```
// Map model data (no copying/parsing)
const tflite::Model* model =
    tflite::GetModel(g_tiny_conv_micro_features_model_data);

// Version check
if (model->version() != TFLITE_SCHEMA_VERSION) {
    error_reporter->Report("Schema version mismatch");
    return 1;
}

// Access subgraph
auto* subgraphs = model->subgraphs();
if (subgraphs->size() != 1) {
    error_reporter->Report("Only 1 subgraph supported");
    return 1;
}
subgraph_ = (*subgraphs)[0];
```

Key Point: Direct memory access, no data structure construction

Tensors and Buffers in FlatBuffers

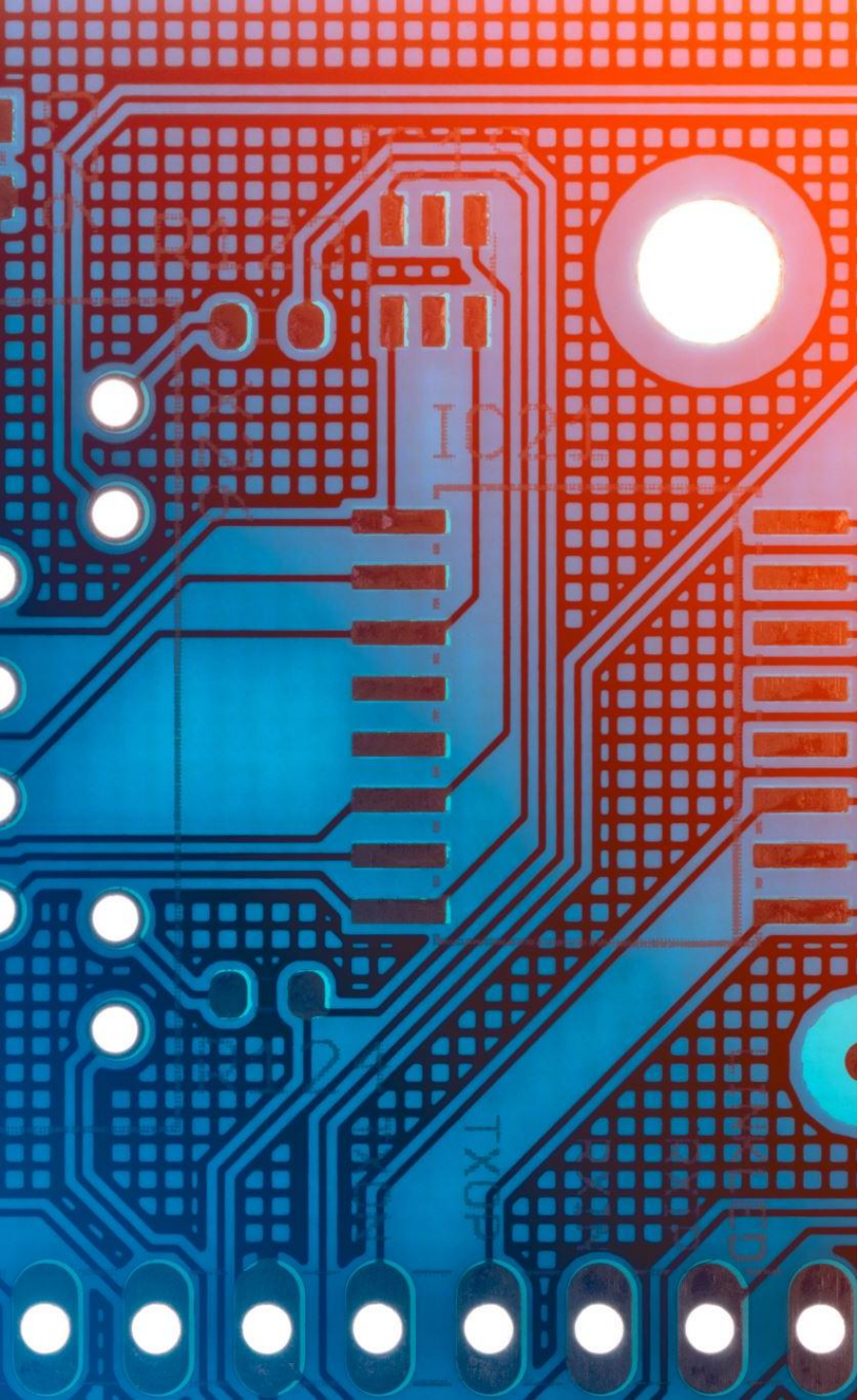
Buffer: Raw data storage

```
table Buffer {  
  data: [ubyte] (force_align: 16); // 16-byte aligned raw data  
}
```

Tensor: Metadata + buffer reference

```
table Tensor {  
  shape: [int]; // Dimensions [batch, height,  
width, channels]  
  type: TensorType; // float32, int8, etc.  
  buffer: uint; // Index into buffers array  
  name: string; // For debugging  
  quantization: QuantizationParameters; // For quantized  
models  
}
```

Separation: Data (buffers) vs Metadata (tensors)



Hardware-Specific Optimizations - Microcontrollers

Microcontroller Optimizations

Objective: Minimize memory and compute usage for resource-constrained devices (e.g., [ARM Cortex-M](#)).

Key Considerations:

- Limited RAM/Flash memory (e.g., KBs of memory).
- No OS or dynamic memory allocators; avoid heap usage.
- Cache misses are costly; optimize access patterns.

Small Data Types:

- Use 8-bit integers (int8) or binary weights instead of FP32 to save memory.
- Quantization is critical (e.g., TensorFlow Lite for Microcontrollers).

Cache-Friendly Access:

- Use linear or sequential memory access to avoid cache thrashing.
- Avoid random jumps (e.g., linked lists).

Static Memory Allocation:

- Pre-allocate buffers during compilation (no malloc at runtime).

Example:

- Replace a float32 dense layer with a int8 quantized layer, reducing memory by 4x.

RAM Usage Analysis

```
// Static allocation approach
uint8_t tensor_arena[50 * 1024]; // 50KB arena

tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    sizeof(tensor_arena), error_reporter);

// Check if arena is sufficient
if (interpreter.AllocateTensors() != kTfLiteOk) {
    // Arena too small - increase size
}
```

Dynamic Memory Requirements

TensorFlow Lite Memory Arena:

The arena holds temporary activation tensors during inference

Arena size calculation:

- `arena_size = max_simultaneous_tensor_memory`

Typical requirements by model type:

- **Simple_dense_model**: 5-20KB
- **Small_CNN**: 20-100KB
- **Efficient_transformer**: 50-200KB

Application Memory:

- Input/output buffers
- Preprocessing workspace
- Communication buffers
- Business logic variables

Memory-Constrained Model Design

Goal: Fit models in limited flash/SRAM while keeping accuracy usable

Budget Planning

- Know flash size (weights/code) and SRAM size (activations/scratch)
- Track peak live activations, not just parameters
- Use static memory plans; preallocate reuse buffers

Design Techniques

- Layer-wise processing
- Run one layer at a time
- Stream weights from flash
- Reuse a single activation buffer across layers

In-place ops

- Use ReLU and similar in-place to avoid extra tensors
- Minimize temporary buffers

Operator fusion

- Fuse Conv + BN + ReLU
- Reduce intermediate tensors and memory traffic

Quantization

- INT8 weights/activations; 32-bit accumulators if needed

Pruning and sparsity

- Remove low-importance channels/connections
- Exploit sparse formats when supported

Shape control

- Cap feature-map sizes (channels/resolution)
- Avoid long skip connections that keep tensors live

Validation

- Measure peak SRAM during inference on device
- Iterate until within flash/SRAM limits